Machine Learning 2 (APM_50441_EP/DSAIB - 2026-2027). Lab5

The objectives of the lab

This lab illustrates the fifth lecture (*Recommender Systems, Text and Words*). Its goals are:

- to build the baselines of a recommender system (global mean, user and item biases) and to see the long tail,
- to implement an item-based collaborative filtering,
- to complete the rating matrix with a low-rank model: SVD, Soft-Impute, alternating least squares and a matrix factorization trained by stochastic gradient descent in PyTorch,
- to evaluate the recommendations with ranking metrics, and to see that a good RMSE does not make a good top-10 list.

The optional *To go further* part, after the core, can be skipped at first: it covers implicit feedback, evaluation issues (time, sampled metrics, exposure bias, feedback loops), cold start and hybrid models, sequential recommendation, and the text half of the lecture (bag of words, BM25, topic models, word vectors, tokenization, RNN and attention).

Use of AI. AI assistants (e.g. the ones built into Colab or VS Code) are allowed, but you must be able to explain every line you use, and answer the *predict before you run* questions before running the code.

Core (about 2 hours)

Exercise 1 (**The data and the baselines**) 1. What is the proportion of observed entries in the 943×1682 rating matrix? Plot the distribution of the ratings, and the number of ratings of each movie sorted in decreasing order (log scale). Which share of the ratings concerns the 10% most rated movies? How many movies have fewer than 5 ratings?

The simplest predictor is the global mean μ of the training ratings. The *bias* model (lecture: *Ratings Issues*) adds a user effect and an item effect, $\hat{r}_{ui} = \mu + b_u + b_i$, fitted by penalized least squares:

$$\min_{b} \sum_{(u,i)\in\Omega} (r_{ui} - \mu - b_u - b_i)^2 + \lambda\Big(\sum_u b_u^2 + \sum_i b_i^2\Big).$$

For fixed item biases, the optimal user bias is $b_u = \frac{\sum_{i\in\Omega_u}(r_{ui}-\mu-b_i)}{|\Omega_u|+\lambda}$, and symmetrically: the problem is solved by alternating these updates (`np.bincount` with weights computes the sums over users or items).

2. Compute the validation RMSE of the global mean. Implement the bias model, choose $\lambda \in \{1, 2, 5, 10, 25\}$ on the validation set, and compare the 5 movies with the highest mean rating with the 5 movies with the highest bias b_i (with their numbers of ratings). Why is the penalty needed? (Store the global mean in `mu` and the biases obtained with $\lambda = 5$ in `b_user` and `b_item`: they are used below.)

Exercise 2 (**Item-based collaborative filtering**)
Collaborative filtering predicts a rating from the ratings of similar items (or users). We work on the *residuals* of the bias model, $\tilde{r}_{ui} = r_{ui} - \mu - b_u - b_i$, stored in a dense matrix `R` with zeros for the missing entries and a boolean mask `W` of the observed ones. The similarity of two items is the cosine of their residual columns. As two movies seen by few common users can get a large similarity by chance, it is *shrunk* towards 0: $s_{ij} \times \frac{n_{ij}}{n_{ij}+\beta}$, n_{ij} being the number of users who rated both.

1. Compute the cosine similarities between the items, with and without the shrinkage ($\beta = 50$). Display the 5 movies most similar to *Star Wars (1977)* in both cases. What does the shrinkage change?

2. Predict the residual $\tilde{r}_{ui}$ by the weighted mean $\sum_{j \in N_k(i,u)} s_{ij} \tilde{r}_{uj} / \sum_j |s_{ij}|$ over the k items rated by u that are the most similar to i, and add it to the bias model. Compute the validation RMSE for $k \in \{10, 30, 100\}$ with the shrunk similarity.

Exercise 3 (**Matrix completion**)

A low-rank model assumes that the matrix of the residuals is close to $UV^\top$, with $U \in \mathbb{R}^{n \times q}$ (the user factors) and $V \in \mathbb{R}^{p \times q}$ (the item factors). The truncated SVD of the zero-filled matrix is the best rank q approximation of `R`, but it treats the missing entries as observed zeros. The weighted problem only fits the observed entries (lecture: *Weighted SVD*); the iterative algorithms below solve it, or its convex relaxation with the nuclear norm (PC5, Exercise 2).

1. Compute the SVD of the zero-filled residual matrix (`np.linalg.svd`), and the validation RMSE of the bias model plus its rank q approximation, for $q \in \{1, 2, 5, 10, 20, 50\}$.

   Hard-Impute fills the missing entries with the current estimate Z and projects on the matrices of rank q: $Z \leftarrow \text{SVD}_q\big(W \odot R + (1 - W) \odot Z\big)$. *Soft-Impute* replaces the projection by a soft thresholding of the singular values, $d_k \mapsto (d_k - \lambda)_+$, which is the proximal operator of $\lambda \| \cdot \|_*$: it solves $\min_Z \frac{1}{2} \|W \odot (R - Z)\|^2 + \lambda \|Z\|_*$ (PC5). A truncated randomized SVD (`randomized_svd`, 100 components) keeps the iterations fast.

2. Write one function performing Hard-Impute (given a rank) or Soft-Impute (given λ), with 20 iterations. Compute the validation RMSE of Hard-Impute for $q \in \{1, 2, 5\}$ and of Soft-Impute along the path $\lambda \in \{40, 20, 15, 10, 7\}$, each solution starting from the previous one (*warm start*); report the rank of each Soft-Impute solution.

Exercise 4 (**Alternating least squares and stochastic gradient descent**)

The factorization can also be fitted directly on the observed entries, with a ridge penalty (lecture: *Regularized Weighted SVD*):

$$\min_{U,V} \sum_{(u,i) \in \Omega} (\tilde{r}_{ui} - U_u^\top V_i)^2 + \lambda \big(\|U\|^2 + \|V\|^2\big).$$

For fixed V, each row U_u is a ridge regression on the items Ω_u rated by u: $U_u = (V_{\Omega_u}^\top V_{\Omega_u} + \lambda I)^{-1} V_{\Omega_u}^\top \tilde{r}_{u,\Omega_u}$ (PC5, Exercise 3), and symmetrically for V. *Alternating least squares* (ALS) alternates these exact updates.

1. Implement ALS (10 iterations, random initialization with standard deviation 0.1), and compute its validation RMSE for $q \in \{5, 10, 20\}$ and $\lambda \in \{10, 20\}$.

   Stochastic gradient descent (the *Funk SVD* of the Netflix prize) updates the factors one rating, or one mini-batch of ratings, at a time: it scales to huge datasets and to any differentiable model. In `PyTorch`, the factors and the biases are *embeddings* (tables of vectors indexed by the users and the items), and the whole model $\hat{r}_{ui} = \mu + b_u + b_i + U_u^\top V_i$ is trained jointly.

2. Write the biased matrix factorization as a `torch.nn.Module` with `nn.Embedding` layers ($q = 20$, factors initialized with standard deviation 0.01, biases with 0), and train it with Adam (learning rate $5 \cdot 10^{-3}$, mini-batches of 1024 ratings, 30 epochs) on the squared loss plus λ times the squared norms of the parameters of the batch. Plot the training and validation RMSE along the epochs for $\lambda \in \{0, 0.1\}$. Which epoch would early stopping choose?

Exercise 5 (**From ratings to recommendations: ranking metrics**)

A recommender system shows a short list of items: only the top of the list matters (lecture: *Ranking Metrics*). For each user, we rank *all* the movies he or she has not rated in the training set (ranking against a few sampled movies may change the comparison of the methods, see *Offline Evaluation Pitfalls*), keep the top $K = 10$, and consider as relevant the movies of the test set rated 4 or 5. The precision $P@K$ and the recall $R@K$ are the fractions of relevant movies in the list and of the relevant movies in the list; NDCG@K discounts the hits by $1/\log_2(1 + \text{rank})$ and normalizes by the best possible value; the MRR is the mean inverse rank of the first hit. The *coverage* is the fraction of the catalog that appears in at least one list.

1. *Predict before you run:* which of the methods has the best NDCG@10? Compute the test RMSE and the ranking metrics of: the popularity (number of training ratings of each movie, the same for every user), the bias model, the ALS model ($q = 10$, $\lambda = 20$) and the PyTorch factorization (trained with $\lambda = 0.1$ for the best number of epochs). Display the top 5 of the first user for each method.

   The rating models predict well how much a user would like a movie *if he or she watched it*, but the test set only contains the movies the users chose to watch and rate. A top-N recommender must also predict *which* movies a user will watch: this is the *implicit feedback* view, where the data are the pairs (u, i) observed, whatever the rating.

2. Build an implicit item-based recommender: from the binary matrix `W` (rated or not), compute the cosine similarities between the items, and score each movie for a user by the sum of its similarities with the movies he or she rated. Compare its ranking metrics with the previous ones.

To go further

The following blocks are optional and independent: each of them only uses the objects defined in the core part (and, for the text blocks, the text data loaded at the beginning of the text part). Their difficulty is marked by stars: ⋆ direct extension, ⋆⋆ new method, ⋆⋆⋆ longer or research-flavored.

Block	Lecture	Stars
User-based collaborative filtering	User-based Filtering	⋆
Similarities for implicit data	Pearson Correlation and Jaccard Index	⋆
Implicit ALS	Weighted Matrix Factorization	⋆⋆
BPR and negative sampling	Negative Sampling	⋆⋆
A temporal split	Offline Evaluation Pitfalls	⋆
Sampled ranking metrics	Offline Evaluation Pitfalls	⋆⋆
Blending	Blending as Stacking	⋆
Cold start: new movies	Possible Solutions, Demographic Approach	⋆⋆
A factorization machine	Factorization Machines and Deep Variants	⋆⋆
A two-tower model	Two-Tower Models	⋆⋆
Exposure bias and inverse propensity scoring	Exposure Bias and Off-Policy Evaluation	⋆⋆
A feedback loop	Feedback Loops and Popularity Bias	⋆⋆
Fairness and diversity	Fairness and Responsible Recommendation	⋆⋆
An A/B test	Metric vs Goals	⋆
Sequential recommendation with a GRU	Sequential Recommendation	⋆⋆⋆
Bag of words and tf-idf	Text and Bag of Words	⋆
A search engine: BM25	Okapi BM25	⋆
Topics: LSA and a multinomial mixture	Multinomial Mixture Model	⋆⋆
Word vectors from co-occurrences	Truncated SVD Approach	⋆⋆
Word2vec: skip-gram with negative sampling	Supervised Learning Formulation	⋆⋆
A BPE tokenizer	Tokenization	⋆⋆
Recurrent networks	Simple Recurrent Unit, Gated Units	⋆⋆
Attention and a small transformer	Transformers	⋆⋆⋆

Exercise 6 (**User-based collaborative filtering** $\star$)
The user-based version predicts the residual $\tilde{r}_{ui}$ from the residuals of the users most similar to u who rated i.

- Compute the shrunk cosine similarities between the users ($\beta = 50$), predict with the k most similar users who rated the movie, and compare the validation RMSE with the item-based version for $k \in \{10, 30, 100\}$. Which similarity matrix is larger? Which one would you precompute for a service with millions of users?

Exercise 7 (**Similarities for implicit data** $\star$)
With binary data, the Jaccard index $|A \cap B|/|A \cup B|$ of the sets of users of two items is a natural similarity. As in the tf-idf weighting of the text part, users who rate many movies are less informative: their co-occurrences can be down-weighted, e.g. by $1/\log(2 + n_u)$.

- Compare the ranking metrics of the implicit item-based recommender with the cosine, the Jaccard index, and the cosine computed with the user weights $1/\log(2 + n_u)$.

Exercise 8 (**Implicit ALS** $\star\star$)
Hu, Koren and Volinsky (2008) factorize the *whole* binary matrix B, unobserved entries included, with confidence weights $c_{ui} = 1 + \alpha B_{ui}$:

$$\min_{X,Y} \sum_{u,i} c_{ui}(B_{ui} - X_u^\top Y_i)^2 + \lambda(\|X\|^2 + \|Y\|^2).$$

All the $n \times p$ entries are in the loss, but the ALS update stays cheap: $X_u = (Y^\top Y + \alpha Y_{\Omega_u}^\top Y_{\Omega_u} + \lambda I)^{-1}(1 + \alpha) Y_{\Omega_u}^\top \mathbf{1}$, where $Y^\top Y$ is computed once per iteration.

- Implement implicit ALS ($q = 64$, $\lambda = 50$, 10 iterations) and compute its ranking metrics for $\alpha \in \{1, 10, 40\}$. Compare with the implicit item-based recommender.

Exercise 9 (**BPR and negative sampling** $\star\star$)
Bayesian personalized ranking (Rendle et al., 2009) learns from pairs: a movie i rated by u should be scored above a movie j drawn at random (a *negative sample*), $\min - \sum \log \sigma(s_{ui} - s_{uj}) + \lambda\|\theta\|^2$ with $s_{ui} = U_u^\top V_i + b_i$. Its expectation over the negatives is a smooth version of $1 -$ AUC of each user's ranking.

- Train BPR in PyTorch ($q = 32$, Adam with learning rate 10^{-2}, mini-batches of 1024 positive pairs, one uniform negative per pair, 20 epochs). Compare its ranking metrics with implicit ALS, and compute the mean AUC of the users on the test set (relevant test movies against all the movies they did not rate).

Exercise 10 (**A temporal split** $\star$)
A random split of the ratings lets the models use the future to predict the past. The *leave-last-out* split keeps, for each user, his or her last rating for the test, as in deployment.

- For each user, hold out either the last rated movie (by timestamp) or a random one, and train the popularity and the implicit item-based recommenders on the other ratings. Compare their hit rate $HR@10$ (is the held-out movie in the top 10?) in both settings.

Exercise 11 (**Sampled ranking metrics** $\star\star$)
To save time, many papers rank each relevant item against 100 random non-rated items only. Krichene and Rendle (2020) showed that such *sampled* metrics can change the order of the methods.

- For the popularity, the bias model, ALS and the implicit item-based recommender, compute the hit rate $HR@10$ of each relevant test movie ranked among 100 random movies not rated by the user, and compare the order of the methods with the full-catalog NDCG@10.

Exercise 12 (Blending ⋆)

The winners of the Netflix prize *blended* more than 100 models: a second-level model (here a linear regression) combines the predictions of the base recommenders, and is trained on held-out ratings (the validation set), as in stacking (Lab 3).

- Fit a linear regression of the validation ratings on the predictions of the bias model, the item kNN ($k = 30$), ALS ($q = 10$, $\lambda = 20$) and the PyTorch factorization, and compare the test RMSE of the blend with the ones of its members.

Exercise 13 (Cold start: new movies ⋆⋆)

A new movie has no rating: collaborative filtering cannot use it. Its *content* (genres, release year) can predict its parameters, here its bias b_i, from the movies already known (a content-based or *hybrid* approach).

- Draw 10% of the movies as "new", remove all their training ratings, and fit the bias model on the others. Predict the bias of the new movies with a ridge regression on the genres and the release year, trained on the known movies. Compare the RMSE on the test ratings of the new movies of: $\mu + b_u$, $\mu + b_u + \hat{b}_i(\text{content})$, and $\mu + b_u + b_i$ fitted with their ratings (the oracle).

Exercise 14 (A factorization machine ⋆⋆)

A factorization machine (Rendle, 2010) is a regression on any sparse features x (one-hot user and item, genres, age, occupation…) with all the pairwise interactions, factorized: $\hat{y} = w_0 + \sum_j w_j x_j + \sum_{j<l} \langle v_j, v_l \rangle x_j x_l$. The interaction term is computed in $O(qd)$ as $\frac{1}{2} \sum_f \left[(\sum_j v_{jf} x_j)^2 - \sum_j v_{jf}^2 x_j^2 \right]$. With only the user and the item, it is the biased matrix factorization.

- Encode each rating by the indices of its active features (user, item, gender, age group, occupation, genres), train a factorization machine ($q = 20$) in PyTorch with the same training loop as the core part, and compare its validation RMSE with the biased factorization.

Exercise 15 (A two-tower model ⋆⋆)

Industrial recommenders first *retrieve* a few hundred candidates among millions of items with a *two-tower* model: a user tower and an item tower map their features to vectors whose dot product is the score, so that the item vectors can be indexed for a fast nearest neighbor search. It is trained on the observed pairs with an *in-batch softmax*: in a batch of pairs (u_k, i_k), each user must recognize its own item among the items of the other pairs. As popular items appear more often in the batches, they are penalized more often as negatives: the *logQ correction* subtracts from each logit the log of the frequency of the item (Yi et al., 2019).

- Train a two-tower model: the user tower embeds the user, gender, age group and occupation, the item tower the movie and its genres, each followed by a linear layer to dimension 32; use the in-batch softmax loss with a temperature 0.1 on mini-batches of 512 training pairs, 10 epochs, without and with the logQ correction. Compute their ranking metrics.

Exercise 16 (Exposure bias and inverse propensity scoring ⋆⋆)

Logged data only concern the items shown by the deployed recommender (the *logging policy* π_0). The value of a new policy π cannot be estimated by the mean reward of the logs, but it can by *inverse propensity scoring*: $\hat{V}_{\text{IPS}}(\pi) = \frac{1}{N} \sum_k \frac{\pi(a_k|x_k)}{\pi_0(a_k|x_k)} r_k$, unbiased if $\pi_0 > 0$ wherever $\pi > 0$ (lecture, after Schnabel et al.). We simulate a world in which the probability that user u likes movie i is $p_{ui} = \sigma(2(\hat{r}_{ui} - 3.5))$, $\hat{r}$ being the ALS predictions.

- Simulate 20 logs of $N = 20\,000$ recommendations: a random user, a movie drawn from the logging policy $\pi_0(i) \propto (\text{number of ratings of } i)^{0.5}$, and a reward drawn with probability p_{ui}. For the new policy $\pi(i|u) \propto \exp(2\hat{r}_{ui})$, compare its true value with the naive estimate (mean reward of the logs), the IPS and the self-normalized IPS estimates (mean and standard deviation over the 20 logs).

Exercise 17 (A feedback loop $\star\star$)

A recommender is trained on what it has shown: popular items get more exposure, hence more interactions, hence more exposure (lecture, after Chaney et al.). Exploration breaks the loop.

- In the simulated world of the previous block (p_{ui} recomputed as there), run 30 rounds in which every user is shown 10 movies and clicks each of them with probability p_{ui}. Compare two systems: a *greedy* one, which shows to everybody the 10 movies with the most clicks so far (starting with random movies), and an *exploring* one, which shows the 8 movies with the best estimated click rate $(\text{clicks}+1)/(\text{shows}+2)$ and 2 random movies. Plot the click rate along the rounds and report the number of distinct movies shown. What does the greedy system learn about the catalog?

Exercise 18 (Fairness and diversity $\star\star$)

Fairness has two sides: the *users* (similar quality for all groups) and the *providers* (a fair exposure of the items). A simple re-ranking divides the scores by a power of the popularity to expose the long tail.

- For the implicit item-based recommender, compute the NDCG@10 by gender and by age group of the users. Then re-rank with the scores divided by $(\text{popularity})^\gamma$ for $\gamma \in \{0, 0.1, 0.2, 0.3\}$ and report the NDCG@10, the coverage and the share of the recommendations going to the 10% most rated movies.

Exercise 19 (An A/B test $\star$)

Offline metrics are only proxies: a new recommender is finally evaluated online, by an A/B test on the metric that matters (clicks, purchases, retention). With a click rate p and n users per group, the difference of the observed rates has a standard deviation $\approx \sqrt{2p(1-p)/n}$.

- How many users per group are needed to detect an increase of the click rate from 5% to 5.25% (a 5% relative lift) with a two-sided test at level 5% and a power of 80%? Check the power by simulating 2000 A/B tests with this size. With $100\,000$ visitors per day, how long does the test last?

Exercise 20 (Sequential recommendation with a GRU $\star\star\star$)

The order of the interactions carries information: what a user watches next depends on what he or she just watched. A sequential recommender (GRU4Rec, SASRec) reads the sequence of the items of a user and predicts the next one, as a language model predicts the next word.

- Sort the ratings of each user by time, hold out the last movie, and train a GRU (embeddings and hidden state of dimension 64, last 50 movies of each sequence) to predict each next movie from the previous ones with the cross-entropy over all the movies (20 epochs, Adam). Compare its $HR@10$ on the held-out movies with the popularity and with the implicit item-based scores of the previous movie only ("users who watched this also watched").

Exercise 21 (Text data (support for the text blocks))

The text blocks use 6 newsgroups of the *20 newsgroups* corpus (messages posted in 1993, without their headers, signatures and quotes), downloaded by `scikit-learn`. The tokenizer keeps the lowercase alphabetic words of at least 2 letters.

Exercise 22 (Bag of words and tf-idf $\star$)

A document is represented by the counts $h_w(d)$ of the words of a vocabulary (the order is lost), weighted by the inverse document frequency $\text{idf}_w = \log(N/N_w)$, which gives no weight to the words appearing everywhere.

- Build the vocabulary of the words appearing in at least 5 documents, the sparse count matrix (`scipy.sparse`) and the tf-idf matrix by hand, and check it against `TfidfVectorizer(smooth_idf=False, norm=None)` (which uses $\log(N/N_w)+1$). Which words have the smallest and the largest idf? Then compare the test accuracy of a logistic regression predicting the newsgroup from the counts, from the tf-idf, and from the tf-idf without the English stop words.

Exercise 23 (A search engine: BM25 ⋆)

Okapi BM25 scores a document d for a query Q by $\sum_{w \in Q} \text{idf}_w \frac{(k_1+1)h_w(d)}{h_w(d)+k_1(1-b+b|d|/\overline{|d|})}$: the term frequency is *saturated* ($k_1 \approx 1.2$) and normalized by the length of the document ($b \approx 0.75$). It is still the reference of sparse retrieval, e.g. in the hybrid search of RAG systems (Lecture 6).

- Implement BM25 on the count matrix of the previous block (rebuild it if needed) and display the newsgroups of the 10 best documents for the queries `"shuttle launch orbit"`, `"goalie penalty playoff"` and `"blood pressure treatment"`. Compare the precision (proportion of the expected newsgroup) of BM25 with the cosine similarity of the tf-idf vectors.

Exercise 24 (Topics: LSA and a multinomial mixture ⋆⋆)

Latent semantic analysis (LSA) is the truncated SVD of the tf-idf matrix; k-means in this space clusters the documents. The *multinomial mixture* model of the lecture is the GMM of text: each document is generated by a topic k (with probability π_k), and its words are drawn independently with the probabilities β_k of this topic; it is fitted by EM (Lab 4).

- Cluster the documents into 6 groups with k-means on the 100 first LSA components (normalized), and with a multinomial mixture fitted by EM (50 iterations, on the counts of the 3000 most frequent words without stop words, in log scale for the E step). Compare their ARI with the newsgroups, and display the 8 most probable words of each topic of the mixture.

Exercise 25 (Word vectors from co-occurrences ⋆⋆)

A word is characterized by the words of its *context* (a window of ± 2 words). The positive pointwise mutual information $\text{PPMI}(w, c) = \max\left(\log \frac{P(w,c)}{P(w)P(c)}, 0\right)$ of the co-occurrence counts, reduced by a truncated SVD, gives dense word vectors (lecture: *Truncated SVD Approach*).

- Count the co-occurrences in a window of ± 2 words between the 5000 most frequent words, compute the PPMI matrix and its truncated SVD with 100 components (word vector $U_w\sqrt{d}$), and display the 6 nearest neighbors (cosine) of `car`, `space`, `doctor`, `gun`, `hockey` and `image`.

Exercise 26 (Word2vec: skip-gram with negative sampling ⋆⋆)

Word2vec turns the co-occurrences into a supervised problem: the true pairs (word, context) are positive examples, pairs with a context drawn at random (with probabilities $\propto$ frequency$^{3/4}$) are negative ones, and $\mathbb{P}(1|w, c) = \sigma(C_w^\top C'_c)$. At the optimum, $C_w^\top C'_c$ is the PMI shifted by the log of the number of negatives (Levy and Goldberg, 2014; PC5).

- Train a skip-gram with negative sampling (5 negatives per pair, vectors of dimension 100, Adam, mini-batches of 4096 pairs, 2 epochs) on the pairs of the previous block (word and context among the 5000 most frequent words), and compare the nearest neighbors with the PPMI vectors.

Exercise 27 (A BPE tokenizer ⋆⋆)

Byte pair encoding (BPE) learns a vocabulary of sub-words: starting from the characters, it repeatedly merges the most frequent pair of adjacent symbols in the corpus. A new word is then split into known sub-words: there is no unknown word.

- Learn 300 BPE merges on the frequencies of the words seen at least 5 times in the corpus (each word being a sequence of characters followed by an end-of-word symbol `_`), and tokenize the words `the`, `space`, `spaceship`, `overfitting`, `unhockeylike` and `hyperparameterization`.

Exercise 28 (Recurrent networks ⋆⋆)

A recurrent network reads a sequence with a hidden state $h_t = \tanh(Wh_{t-1} + Ux_t + b)$. Back-propagation

through time multiplies Jacobians: the gradient of h_T with respect to h_{T-k} vanishes or explodes geometrically with k, depending on the scale of W. Gated units (LSTM, GRU) add an *additive* memory path that keeps the gradient alive.

- For a simple recurrent unit with $W = \rho Q$ (Q a random orthogonal 64×64 matrix, $\rho \in \{0.8, 1, 1.25\}$, random inputs), compute with `torch.autograd` the norm of $\partial h_{60}/\partial h_{60-k}$ (applied to a random direction) for k from 1 to 50 and plot it in log scale. Then train an LSTM classifier of the newsgroups (vocabulary of the $10\,000$ most frequent words, embeddings of dimension 64, 64 hidden units, first 100 words of each document, 4 epochs) and compare its test accuracy with the bag of words.

Exercise 29 (**Attention and a small transformer** $\star\star\star$)

Self-attention mixes the vectors of a sequence $X \in \mathbb{R}^{T \times d}$ with weights computed from the data: $\text{softmax}\big(QK^\top/\sqrt{d_k}\big)V$ with $Q = XW_Q$, $K = XW_K$, $V = XW_V$. It is *permutation equivariant*: without positional encodings, a transformer sees a bag of words (PC5). All the positions are processed in parallel, unlike a recurrent network.

- Implement a single-head self-attention by hand and check that permuting its input permutes its output. Then train a small transformer classifier (`torch.nn.TransformerEncoder`, 1 layer, dimension 64, 4 heads, learned positional embeddings, mean pooling over the non-padded positions, 4 epochs) on the same sequences (100 first words), and compare its test accuracy with the LSTM and the bag of words.

Use of AI. Artificial intelligence tools have been used to revise the content of this lab. The main contributor takes full responsibility for this content.

License. The texts of this lab are under the Creative Commons Attribution-ShareAlike 4.0 license (CC BY-SA 4.0), and its code under the MIT license; the datasets keep their own licenses. Main contributor: E. Le Pennec. Contributors: S. Boucheron, A. Dieuleveut, A.K. Fermin, S. Gadat, S. Gaiffas, B. Goujaud, A. Guilloux, Ch. Keribin, Th. Klein, E. Matzner, J. Read, M. Sangnier, E. Scornet.